MATLAB Assignment 4

Farhan Arnab

Department: Nuclear Engineering
Course code: 3204
Roll: 03 [EK-076-003]

Contents

1 Number 1(a) 2
- 1.1 Theory . . . 2
- 1.2 Algorithm . . . 2
- 1.3 Matlab Code . . . 4
- 1.4 Output & Results . . . 5
- 1.5 Discussion . . . 6

2 Number 1(b) 7
- 2.1 Theory . . . 7
- 2.2 Algorithm . . . 7
- 2.3 Matlab Code . . . 8
- 2.4 Output & Results . . . 10
- 2.5 Discussion . . . 11

3 Functions 12
- 3.1 myJacobi . . . 12
- 3.2 mySeidal . . . 12
- 3.3 mySOR . . . 13

4 References 14

1 Number 1(a)

1.1 Theory

For the steady state case, we drop the temporal term and we are left with

$$\frac{\partial^2 \xi}{\partial x^2} + \frac{\partial^2 \xi}{\partial y^2} = 0$$

To solve this numerically we can use a meshgrid and use finite difference to replace the partial derivatives in the space coordinates. Applying central difference to both

$$\frac{\Xi^n_{i-1,j} - 2\Xi_{i,j} + \Xi^n_{i+1,j}}{(\Delta x)^2} + \frac{\Xi^n_{i,j-1} - 2\Xi_{i,j} + \Xi^n_{i,j+1}}{(\Delta y)^2} = 0$$

If we take $\Delta x = 0.1$ and $\Delta y = 0.05$ we get

$$\Xi^n_{i-1,j} + \Xi^n_{i+1,j} + 4\Xi^n_{i,j-1} + 4\Xi^n_{i,j+1} - 10\Xi^n_{i,j1} = 0 \qquad i = 1, 2, ...9, j = 1, 2, ...19$$

The meshgrid in figure 1 shows the method of numbering of the points in the mesh, excluding the outer boundary.

1.2 Algorithm

1. First, a meshgrid is set up using the proper stepsizes.
2. A 171×171 matrix is set up with to represent the linear set of equations for each point under consideration in the meshgrid (a coefficient matrix). Similarly the constant matrix is also setup such that

   $$\boldsymbol{A\Xi} = \boldsymbol{B}$$

   where, $\boldsymbol{\Xi} = \begin{bmatrix} \Xi_1 & \Xi_2 & \Xi_3 & \Xi_4 & \dots & \Xi_{171} \end{bmatrix}^\intercal$.
3. Self coded solvers can be used or the built in backslash operator to solve this matrix problem.
4. The solution matrix obtained is then reshaped and accordingly inputted into another matrix so that it gives the flux to remain consistent with the meshgrid previously created.

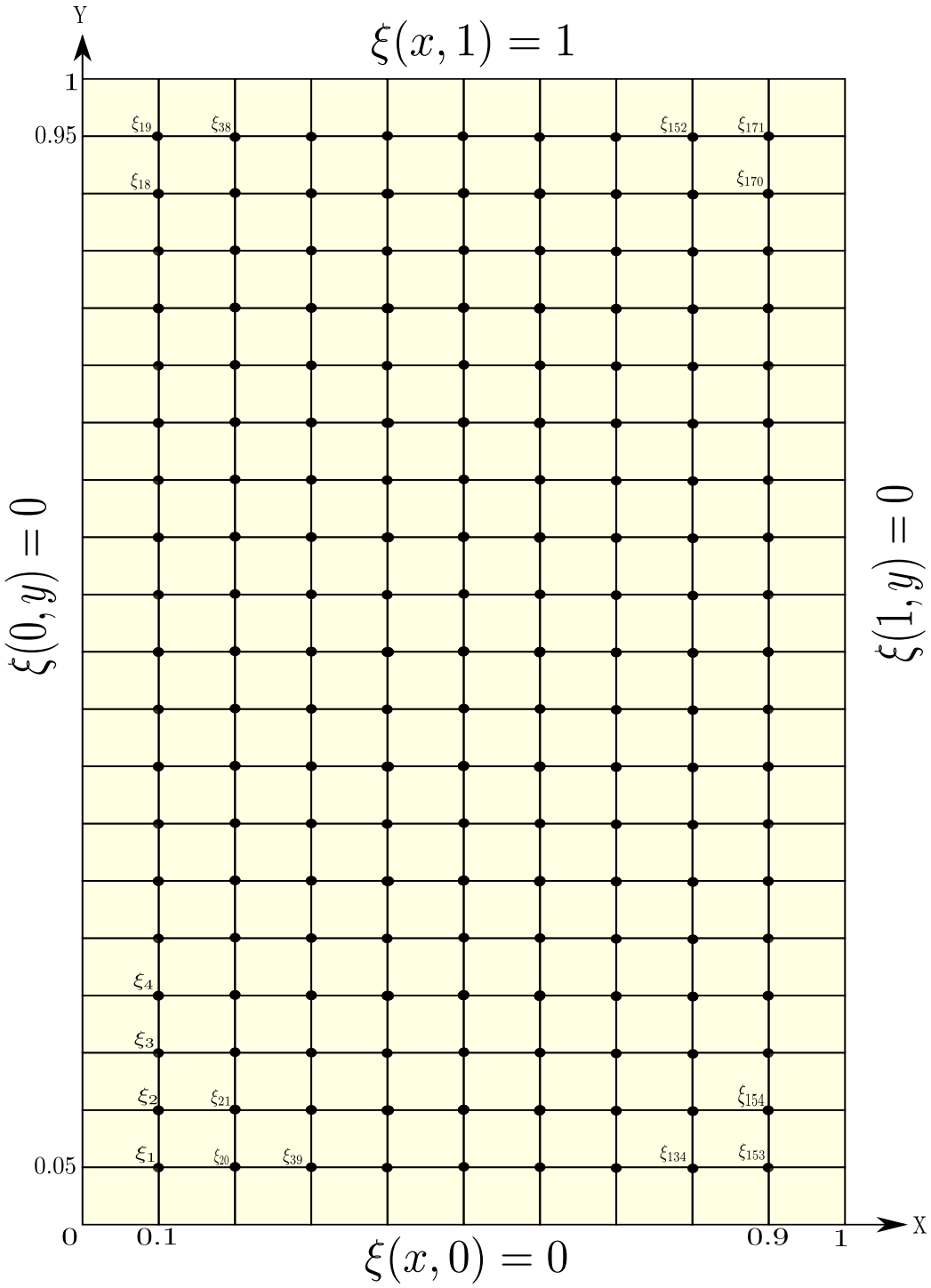


Figure 1: Meshgrid used

1.3 Matlab Code

```
clc; clear;

[x1, y1] = meshgrid(0:0.1:1, 0:0.05:1);

%coefficient matrix
A1 = zeros(171);
A1 = A1 + diag(-10*ones(1, 171)) +...
 4*diag(ones(1,170), 1) + 4*diag(ones(1,170), -1);
A1 = A1 + diag(ones(1,171-19), 19) + diag(ones(1,171-19),...
 -19);

for i = 19:19:152
    A1(i,i+1) = 0;
    A1(i+1,i) = 0;
end

%constant matrix
B1 = zeros(171, 1);
for i = 19:19:171
    B1(i) = -4;
end

[E1, ~] = mySOR(A1,B1,1.1);

%solution matrix according to the shape of the mesh
Z1 = zeros(21, 11);
Z1(21, :) = 1;

for i = 1:9
    Z1(2:20, i+1) = E1(19*i-18:19*i);
end

contour(x1, y1, Z1, LineStyle="-.", FaceAlpha=0.4,...
 FaceColor="auto")
yticks(0:0.1:1)

[~, iteration_jacobi] = myJacobi(A1,B1);
fprintf('It took %d iterations for the Jacobi solver to...
```

```
 converge\n', iteration_jacobi);

[~, iteration_seidal] = mySeidel(A1, B1);
fprintf('It took %d iterations for the Seidal solver to...
 converge\n', iteration_seidal);

[~, iteration_SOR] = mySOR(A1, B1, 1.1);
fprintf('It took %d iterations for the SOR solver to...
 converge\n', iteration_SOR);
```

1.4 Output & Results

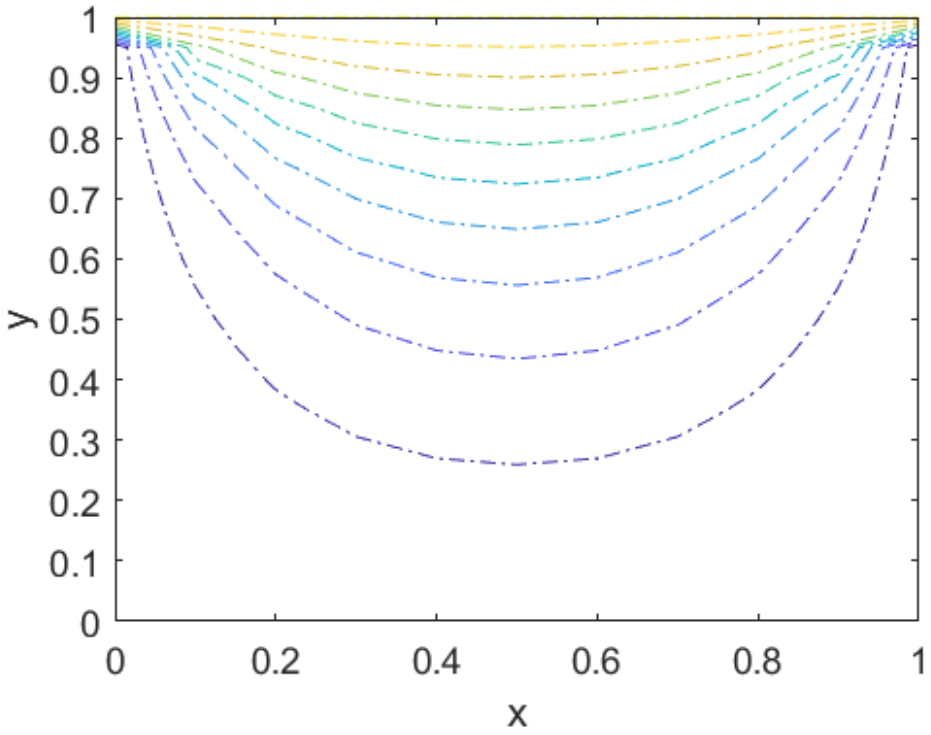


Figure 2: Contour of steady state conditions

```
It took 559 iterations for the Jacobi solver to converge
```

```
It took 302 iterations for the Seidal solver to converge
It took 252 iterations for the SOR solver to converge
```

1.5 Discussion

The steady state solution obtained is actually the transient state solution as $t \to \infty$. The plot obtained is slightly rough around some edges, which can easily be smoothed by taking smaller steps in space, i.e. a finer meshgrid. It is also observed that *mySOR* function converges much quicker as demonstrated by the number of iterations taken.

2 Number 1(b)

2.1 Theory

For the unsteady state equation we have the following

$$\frac{\partial \xi}{\partial t} + \frac{\partial^2 \xi}{\partial x^2} + \frac{\partial^2 \xi}{\partial y^2} = 0$$

Similar to section 1.1 we replace the partial derivatives with their finite difference formulas. Using central difference in space and forward difference in time we get

$$\frac{\Xi_{i,j}^{n+1} - \Xi_{i,j}^{n}}{\Delta t} + \frac{\Xi_{i-1,j}^{n} - 2\Xi_{i,j} + \Xi_{i+1,j}^{n}}{(\Delta x)^2} + \frac{\Xi_{i,j-1}^{n} - 2\Xi_{i,j} + \Xi_{i,j+1}^{n}}{(\Delta y)^2} = 0$$

$$\Xi_{i,j}^{n+1} = \Xi_{i,j}^{n} + \frac{\Delta t}{(\Delta x)^2}\left(\Xi_{i-1,j}^{n} - 2\Xi_{i,j} + \Xi_{i+1,j}^{n}\right) + \frac{\Delta t}{(\Delta y)^2}\left(\Xi_{i,j-1}^{n} - 2\Xi_{i,j} + \Xi_{i,j+1}^{n}\right)$$

Again the meshgrid in figure 1 is used with the range of values of i and j the same as in section 1.1.

2.2 Algorithm

1. First, a meshgrid is setup using the proper stepsizes.
2. Instead of using a single matrix to solve a set of linear equations at a single go, we can use an iterative approach. Starting with an initial condition of $\Xi = 0$ at every coordinate except at $y = 1$ where $\Xi = 1$, we setup a three dimensional matrix. The third dimension is for the time; at each timestep, we move up one level in the third dimension of the matrix.
3. The number of timesteps is arbitrarily chosen, the code is written in a general way such that easy modifications are possible if the user wants it.
4. Solvers in section 3 are not required since this is a simple iteration with time.
5. A *for* loop is used with a small pause to get an animation of the contour at different times.
6. A plot of the absolute error between the time steps is also drawn at specific times.

2.3 Matlab Code

```
clear
clc
close all

%spacesteps and meshgrid
dx = 0.1;
dy = 0.05;
[X, Y] = meshgrid(0:dx:1, 0:dy:1);

%timestep
dt = 0.001;
final_time = 0.250;
n = final_time/dt + 1;

%solution matrix with third dimension for time
Z = zeros(1/dy+1,1/dx+1,n);
Z(end,:,:) = 1;

for t = 1:n-1
    for i = 2:1/dx
        for j = 2:1/dy
            Z(j,i,t+1) = Z(j,i,t) + (dt/(dx^2))...
             * (Z(j,i+1,t) - 2*Z(j,i,t) + Z(j,i-1,t))...
              + (dt/(dy^2)) * (Z(j+1,i,t) - 2*Z(j,i,t)...
               + Z(j-1,i,t));
        end
    end
end

%plot at each time step
figure (1);
for t = 1:n
    contour(X,Y,Z(:,:,t),12,LineStyle="--",...
     FaceColor="auto");
    colorbar
    str = sprintf('Time = %.3f sec', t*dt-dt);
    title(str);
    xlabel('x')
```

```
    ylabel('y')
    pause(0.005)
end

error = zeros(1,n-1);
for i = 1:n-1
    error(i) = max(abs(Z(:,:,i+1) - Z(:,:,i)), [], "all");
end

%error plot
figure (2);
plot(0.001:0.001:0.25, error)
xlabel('Time');
ylabel('Absolute error');

%plot at 4 different times for report
figure (3)
subplot(2,2,1)
contour(X,Y,Z(:,:,21),13)
str = sprintf('Time = %.3f sec', 0.02);
title(str);
xlabel('x')
ylabel('y')

subplot(2,2,2)
contour(X,Y,Z(:,:,51),13)
str = sprintf('Time = %.3f sec', 0.05);
title(str);
xlabel('x')
ylabel('y')

subplot(2,2,3)
contour(X,Y,Z(:,:,101),13)
str = sprintf('Time = %.3f sec', 0.1);
title(str);
xlabel('x')
ylabel('y')

subplot(2,2,4)
contour(X,Y,Z(:,:,251),13)
```

```
str = sprintf('Time = %.3f sec', 0.25);
title(str);
xlabel('x')
ylabel('y')
```

2.4 Output & Results

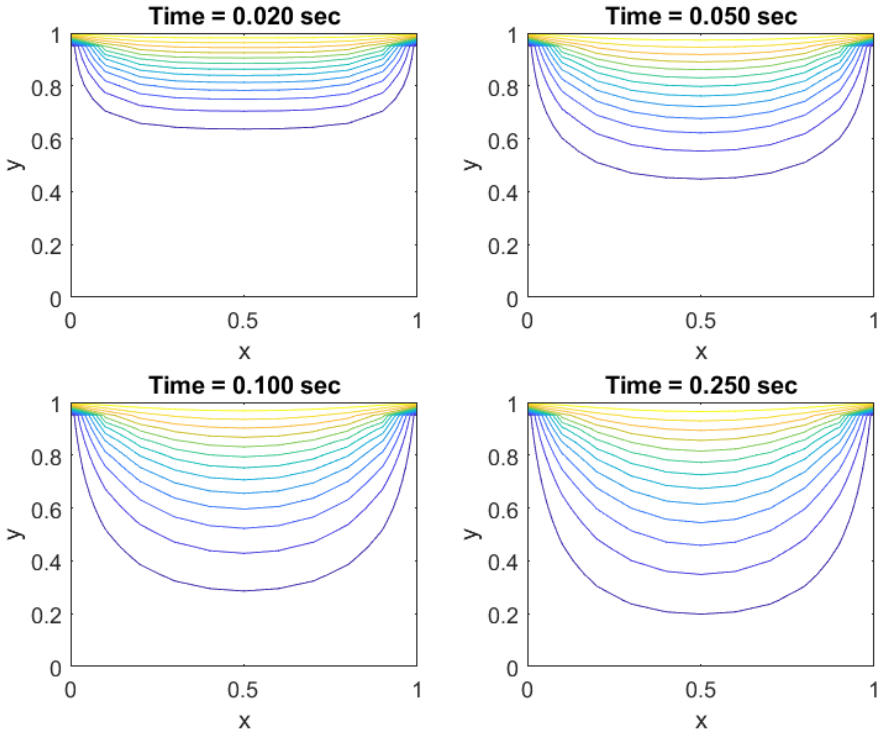


Figure 3: Contour of unsteady state at different times

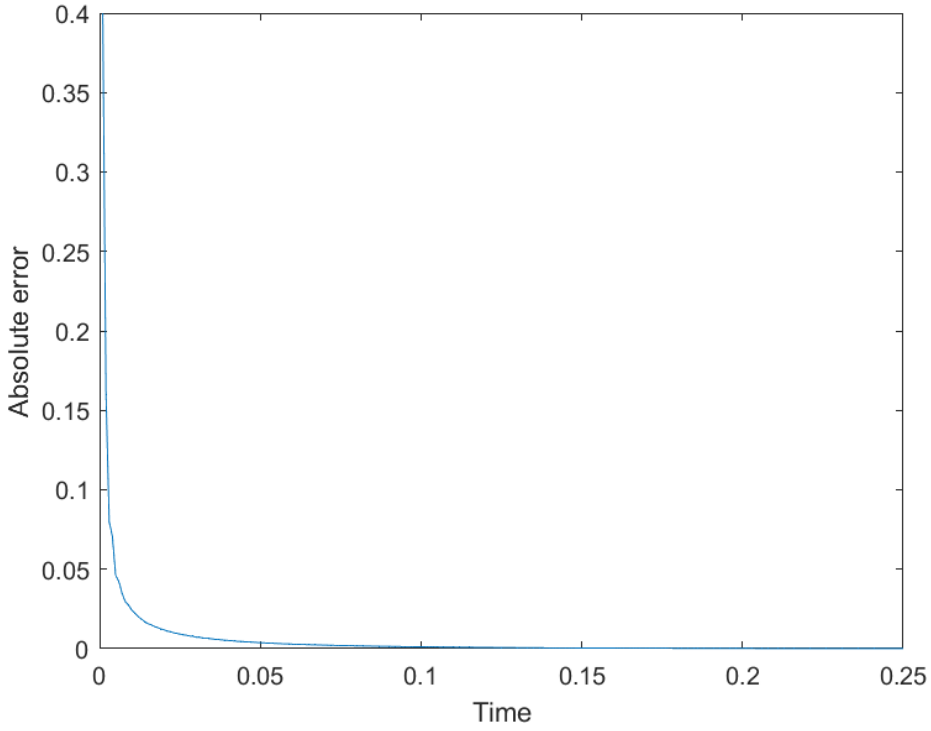


Figure 4: Absolute error at different times

2.5 Discussion

An animation was obtained for contours at each value of time displaying the flux to gradually reach the steady state condition obtained in the previous section. Since this cannot be shown in this report, another plot is given for contours at 4 different times. Again, decreasing the space stepsize can smooth the contours. However it is noted that smaller stepsizes in space requires a smaller timestep to meet the convergence criteria of the explicit method used.
The plot of absolute error exhibit a sharp decrease in error in the first 0.025 seconds and gradually flattening out as the solution converges at around 0.15 seconds.

3 Functions

3.1 myJacobi

```
function [out, iteration] = myJacobi(A, B)
%function uses Jacobi iteration with to solve a linear set
% of equations
%in the form of Ax=b. Pass the matrix A and B to output the
% solution matrix
%and the number of iteration.

    len = length(B);
    sol = zeros(1, len);
    error = 100;
    iteration = 0;

    while error>1e-5
        temp_out = sol;
        for i = 1:len
            temp_coeff = A(i, :);
            temp_coeff(i) = [];
            temp_sol = sol;
            temp_sol(i) = [];
            temp_out(i) = (B(i) - sum(temp_sol.*temp_coeff...
            ))/A(i,i);
        end
        error = sum(abs(temp_out - sol));
        sol = temp_out;
        iteration = iteration+1;
    end
    out = sol;
end
```

3.2 mySeidal

```
function [out, iteration] = mySeidel(A, B)
%function uses Gauss Seidal iteration with to solve a linear
%set of
% equations in the form of Ax=b. Pass the matrix A and B to
```

```
% output the
%solution matrix and the number of iteration.

    len = length(B);
    sol = zeros(1, len);
    sol_ref = sol;
    error = 100;
    iteration = 0;

    while error>1e-5
        for i = 1:len
            temp_coeff = A(i, :);
            temp_coeff(i) = [];
            temp_sol = sol;
            temp_sol(i) = [];
            sol(i) = (B(i) - sum(temp_sol.*temp_coeff)...
            )/A(i,i);
        end
        error = sum(abs(sol - sol_ref));
        sol_ref = sol;
        iteration = iteration+1;
    end

    out = sol;
end
```

3.3 mySOR

```
function [out, iteration] = mySOR(A, B, w)
%function uses SOR iteration with to solve a linear set of
%equations
%in the form of Ax=b. Pass the matrix A and B to output the
% solution matrix
%and the number of iteration. The value of w is also passed
%to get the
%overshoot or undershoot

    len = length(B);
    sol = zeros(1, len);
    sol_ref = sol;
```

```
    error = 100;
    iteration = 0;

    while error>1e-5
        for i = 1:len
            temp_coeff = A(i, :);
            temp_coeff(i) = [];
            temp_sol = sol;
            temp_sol(i) = [];
            sol(i) = (1-w)*sol(i) + w*(B(i) - sum(...
            temp_sol.*temp_coeff))/A(i,i);
        end
        error = sum(abs(sol - sol_ref));
        sol_ref = sol;
        iteration = iteration + 1;
    end
    out = sol;
end
```

4 References

[1] Elementary Numerical Analysis, Kendall Atkinson, John & Wiley Sons Inc, 2004